In [1]:
```
#%pip install torch torchvision pillow scikit-learn matplotlib
```

In [ ]:
```
import pandas as pd
import numpy as np

import os
from pathlib import Path

from PIL import Image
from torchvision import transforms

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import models

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score

import matplotlib.pyplot as plt
```

In [3]:
```
#Load data
data_df = pd.read_csv('combined_reduced.csv')
```

In [4]:
```
data_df.isnull().sum()
```

Out[4]:
```
image_id               0
location_id            0
image_date_time        0
species_common_name    0
species_group          0
media_url              0
filename               0
dtype: int64
```

In [5]:
```
# Convert labels to coded integers

class_to_idx = {
    "birds": 0,
    "large carnivore": 1,
    "small mammals": 2,
    "ungulates": 3,
}

data_df["label"] = data_df["species_group"].map(class_to_idx)
data_df["label"].unique()
#data_df["label"] = data_df["label"].astype(int)
```

Out[5]:
```
array([3, 1, 2, 0])
```

In [6]:
```
data_df.head()
```

Out[6]:

	image_id	location_id	image_date_time	species_common_name	species_group	media_url	filename	label
0	155562171	524567	2023-01-27 00:51:53	Moose	ungulates	https://wildtrax-cam-ca.s3-accelerate.amazonaw...	155562171.jpg	3
1	155562172	524567	2023-01-27 00:51:55	Moose	ungulates	https://wildtrax-cam-ca.s3-accelerate.amazonaw...	155562172.jpg	3
2	155562173	524567	2023-01-27 00:51:57	Moose	ungulates	https://wildtrax-cam-ca.s3-accelerate.amazonaw...	155562173.jpg	3
3	155562174	524567	2023-01-27 00:52:10	Moose	ungulates	https://wildtrax-cam-ca.s3-accelerate.amazonaw...	155562174.jpg	3
4	155562175	524567	2023-01-27 00:52:12	Moose	ungulates	https://wildtrax-cam-ca.s3-accelerate.amazonaw...	155562175.jpg	3

In [7]:
```
#filter out of the csv any images that are linked to but not in the image folder
```

```python
import os

data_df["image_path"] = "C:/Users/mijim/Desktop/images/" + data_df["filename"]

data_df["exists"] = data_df["image_path"].apply(os.path.exists)

print(data_df["exists"].value_counts())

data_df = data_df[data_df["exists"]].copy()
```

```
exists
True     24732
False        5
Name: count, dtype: int64
```

In [8]:

```python
data_df = data_df[["image_path", "label"]]
data_df.dropna(inplace=True) #remove the rows without labels

data_df.head()
```

Out[8]:

	image_path	label
0	C:/Users/mijim/Desktop/images/155562171.jpg	3
1	C:/Users/mijim/Desktop/images/155562172.jpg	3
2	C:/Users/mijim/Desktop/images/155562173.jpg	3
3	C:/Users/mijim/Desktop/images/155562174.jpg	3
4	C:/Users/mijim/Desktop/images/155562175.jpg	3

In [9]:

```python
from sklearn.model_selection import train_test_split

# data90, data10 = train_test_split(
#     data_df,
#     test_size=0.1,
#     random_state=42,
#     stratify=data_df["label"]
# )

train_df, temp_df = train_test_split(
```

```
    data_df, #data10 when testing large numbers of files
    test_size=0.3,
    random_state=42,
    stratify=data_df["label"]
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.5,
    random_state=42,
    stratify=temp_df["label"]
)
```

In [10]:

```
#check class balance
print("Train distribution:")
print(train_df["label"].value_counts(normalize=True))

print("\nValidation distribution:")
print(val_df["label"].value_counts(normalize=True))

print("\nTest distribution:")
print(test_df["label"].value_counts(normalize=True))
```

```
Train distribution:
label
3    0.724584
1    0.223602
2    0.041359
0    0.010455
Name: proportion, dtype: float64

Validation distribution:
label
3    0.724798
1    0.223720
2    0.041240
0    0.010243
Name: proportion, dtype: float64

Test distribution:
label
3    0.724528
1    0.223450
2    0.041509
0    0.010512
Name: proportion, dtype: float64
```

In [11]:
```
transform = transforms.Compose([  #set the image transformer
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

In [12]:
```
class WildlifeDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df = dataframe.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        image_path = row["image_path"]
        label = int(row["label"])   # ensure it's an int
```

```
        try:
            image = Image.open(image_path).convert("RGB") #open the image and convert it to RGB
        except:
            # fallback: skip or handle
            return self.__getitem__((idx + 1) % len(self.df))

        if self.transform:
            image = self.transform(image)

        return image, label
```

In [13]:
```
train_dataset = WildlifeDataset(train_df, transform=transform)
val_dataset = WildlifeDataset(val_df, transform=transform)
test_dataset = WildlifeDataset(test_df, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

In [14]:
```
#Before training check the shape of the datasets
#check TRAIN dataset
train_dataset = WildlifeDataset(train_df, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=0)

images, labels = next(iter(train_loader))

print("Train batch:", images.shape, labels.shape)   # should be [32, 3, 224, 224]

#validation dataset
val_dataset = WildlifeDataset(val_df, transform=transform)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_workers=0)

images, labels = next(iter(val_loader))
print("Val batch:", images.shape, labels.shape)

#test dataset
test_dataset = WildlifeDataset(test_df, transform=transform)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=0)
```

```
images, labels = next(iter(test_loader))
print("Test batch:", images.shape, labels.shape)
```

```
Train batch: torch.Size([32, 3, 224, 224]) torch.Size([32])
Val batch: torch.Size([32, 3, 224, 224]) torch.Size([32])
Test batch: torch.Size([32, 3, 224, 224]) torch.Size([32])
```

In [15]:

```
num_classes = 4
class_names = ["birds", "large carnivore", "small mammals", "ungulates"]

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  #attempt to use GPU

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT) #assign the model

#freeze the model -- stop learning. we're using it pre-trained
for param in model.parameters():
    param.requires_grad = False


model.fc = nn.Linear(model.fc.in_features, num_classes) #replace the final layer. model.fc is a variable in the model
model = model.to(device)
```

In [16]:

```
# look at what the model (resnet) is composed of

print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer3): Sequential(
```

```
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=4, bias=True)
)
```

In [17]:

```
#USE WEIGHTS TO HANDLE CLASS IMBALANCE
class_props = train_df["label"].value_counts(normalize=True).sort_index()
class_props = torch.tensor(class_props.values, dtype=torch.float32)  # proportions
weights = 1.0 / class_props
weights = weights / weights.sum() * len(weights)
weights = weights.to(device)

loss_function = nn.CrossEntropyLoss(weight=weights) #need to specify loss. use this standard loss. apply weights
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)  #need to specify an optimizer. Adam is pretty standard.

num_epochs = 10

train_losses = []
val_losses = []

train_accuracies = []
val_accuracies = []

for epoch in range(num_epochs):

    # --- TRAIN ---
    model.train() #put into training mode
    running_loss = 0.0
    total = 0
    correct = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        loss = loss_function(outputs, labels)

        optimizer.zero_grad() #reset gradients
        loss.backward()         #compute gradients
        optimizer.step()        #update model

        running_loss += loss.item() * images.size(0)
        total += labels.size(0)

        preds = outputs.argmax(dim=1)
```

```python
        correct += (preds == labels).sum().item()

    epoch_train_loss = running_loss / total
    epoch_train_acc = correct / total
    train_losses.append(epoch_train_loss)
    train_accuracies.append(epoch_train_acc)

    print(f"Epoch {epoch+1}/{num_epochs}")
    print("Train loss:", epoch_train_loss)
    print("Train accuracy:", epoch_train_acc)

        # --- VALIDATION ---
    model.eval() #put into eval mode
    val_loss = 0.0
    val_total = 0
    val_correct = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = loss_function(outputs, labels)

            val_loss += loss.item() * images.size(0)
            val_total += labels.size(0)

            preds = outputs.argmax(dim=1)
            # move to CPU + convert to numpy
            all_preds.extend(preds.cpu().numpy()) # running total of predictions. required for confusion matrix
            all_labels.extend(labels.cpu().numpy()) # running total of true labels. required for confusion matrix
            val_correct += (preds == labels).sum().item()

    epoch_val_loss = val_loss / val_total
    epoch_val_acc = val_correct / val_total

    val_losses.append(epoch_val_loss)
    val_accuracies.append(epoch_val_acc)
```

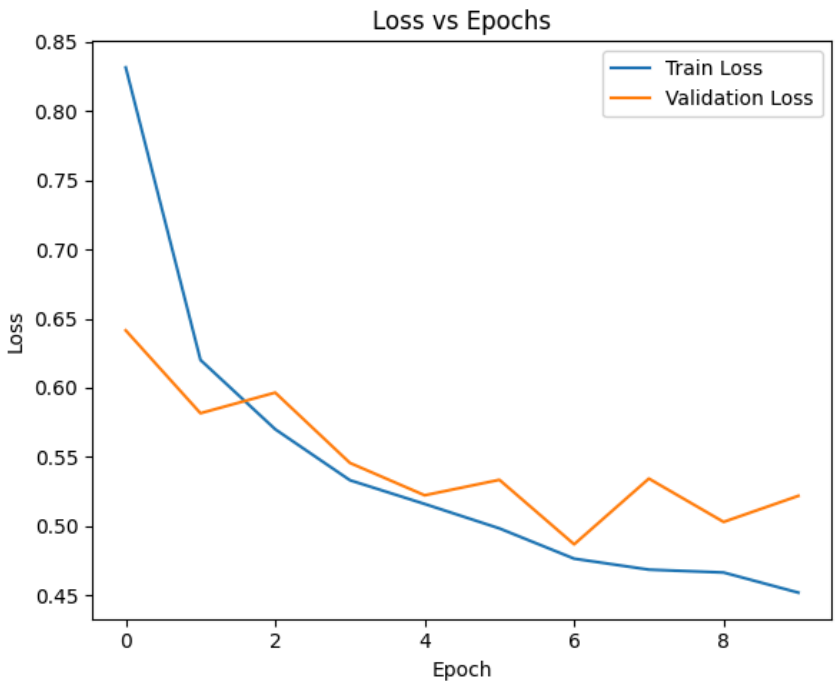
```
    print("Val loss:", epoch_val_loss)
    print("Val accuracy:", epoch_val_acc)
```

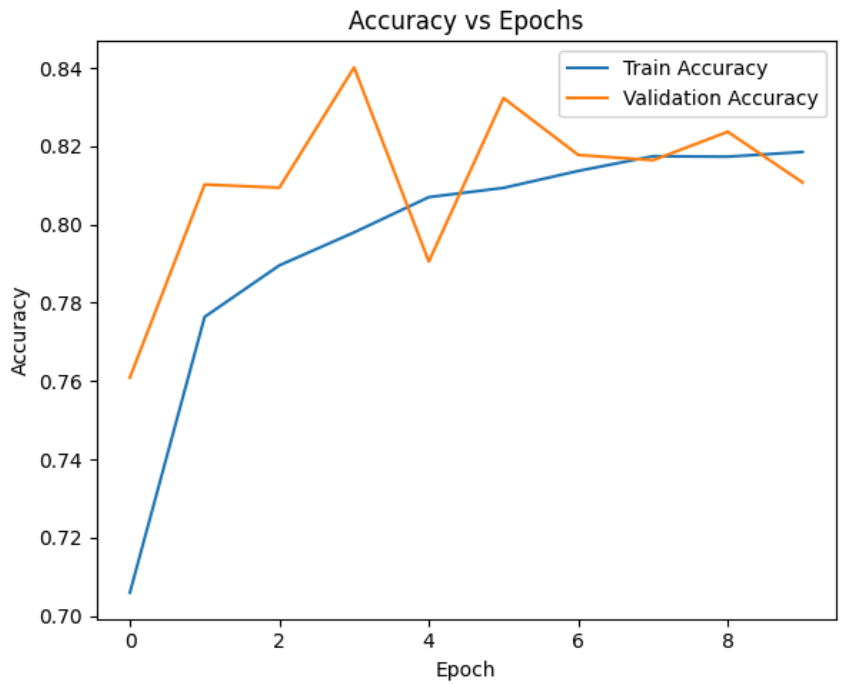
```
Epoch 1/10
Train loss: 0.831361457788129
Train accuracy: 0.7059265249537893
Val loss: 0.6414569705001749
Val accuracy: 0.7609164420485175
Epoch 2/10
Train loss: 0.6201142959310475
Train accuracy: 0.7763978743068392
Val loss: 0.5816442033351271
Val accuracy: 0.8102425876010781
Epoch 3/10
Train loss: 0.570080889901921
Train accuracy: 0.7895679297597042
Val loss: 0.5965257892068827
Val accuracy: 0.809433962264151
Epoch 4/10
Train loss: 0.5332639558643598
Train accuracy: 0.7980013863216266
Val loss: 0.5457402252444038
Val accuracy: 0.8401617250673854
Epoch 5/10
Train loss: 0.5161105614575573
Train accuracy: 0.8070124768946395
Val loss: 0.5223834994025629
Val accuracy: 0.7905660377358491
Epoch 6/10
Train loss: 0.498468138869721
Train accuracy: 0.8093807763401109
Val loss: 0.5335458463735657
Val accuracy: 0.832345013477089
Epoch 7/10
Train loss: 0.4766077906471744
Train accuracy: 0.8137130314232902
Val loss: 0.4869829621919082
Val accuracy: 0.8177897574123989
Epoch 8/10
Train loss: 0.4686923236370968
Train accuracy: 0.8174676524953789
Val loss: 0.534430684105084
Val accuracy: 0.8164420485175202
Epoch 9/10
Train loss: 0.46660717166340504
```

```
Train accuracy: 0.8173521256931608
Val loss: 0.5031488218718784
Val accuracy: 0.8237196765498652
Epoch 10/10
Train loss: 0.4520862385933589
Train accuracy: 0.8185651571164511
Val loss: 0.5219566187125975
Val accuracy: 0.8107816711590297
```

In [27]:

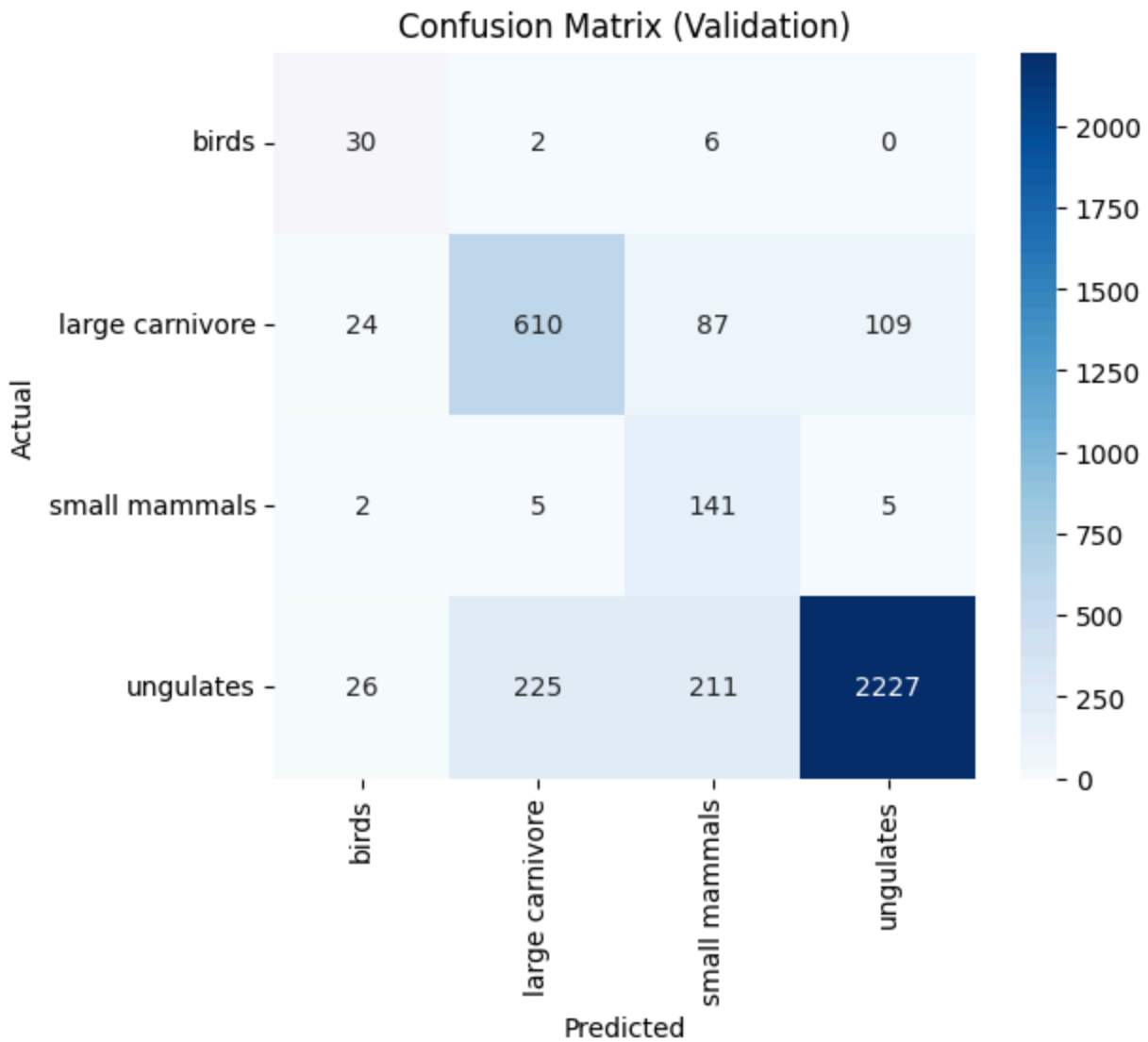
```python
import matplotlib.pyplot as plt

# multiple columns plot
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# loss
axes[0].plot(train_losses, label="Train Loss")
axes[0].plot(val_losses, label="Validation Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].set_title("Loss vs Epochs")
axes[0].legend()

# acc
axes[1].plot(train_accuracies, label="Train Accuracy")
axes[1].plot(val_accuracies, label="Validation Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].set_title("Accuracy vs Epochs")
axes[1].legend()

plt.tight_layout()

plt.show()
```





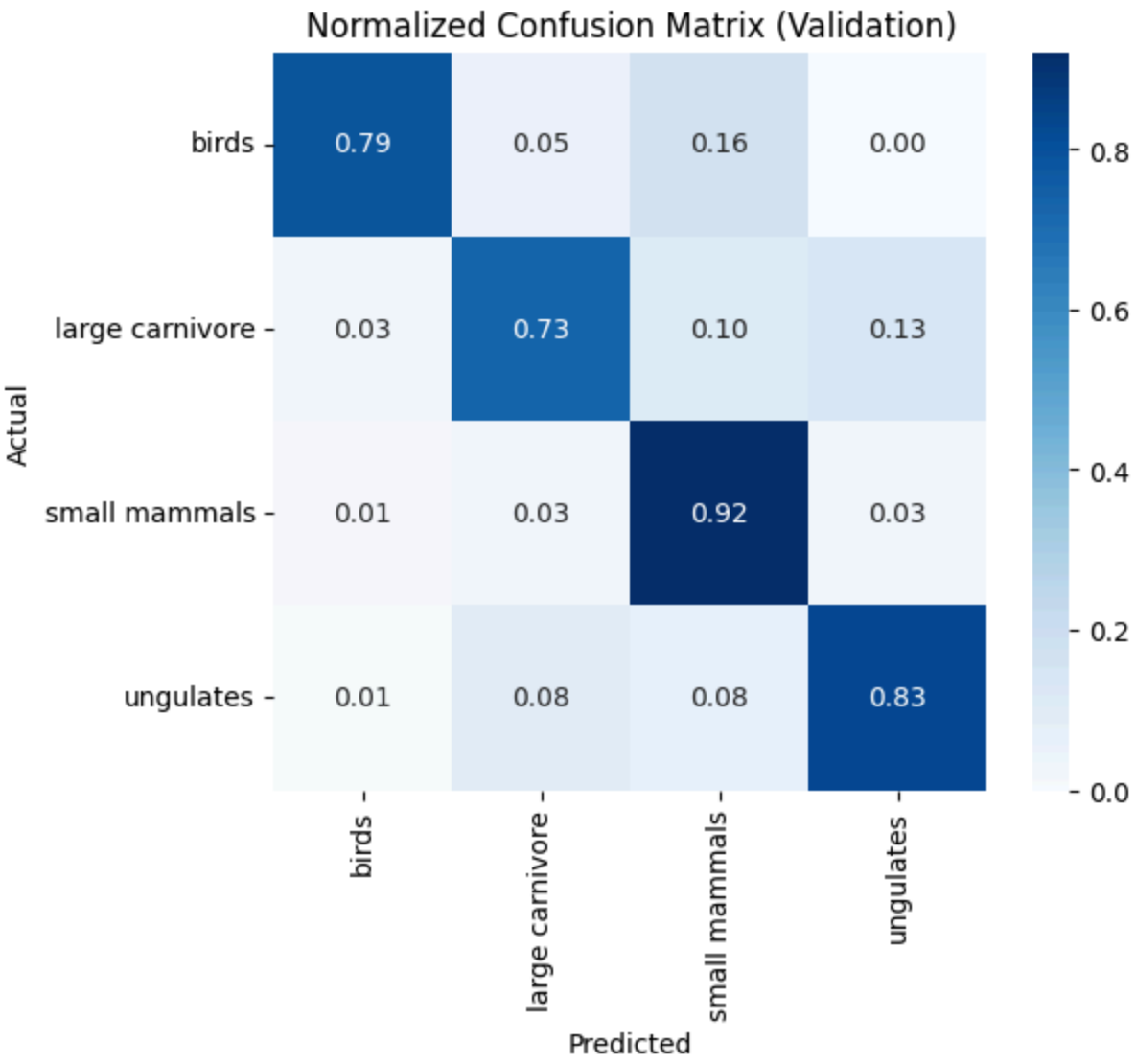
In [20]:

```
# validation confusion matrix

cm = confusion_matrix(all_labels, all_preds)

import seaborn as sns

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (Validation)")
plt.show()
```



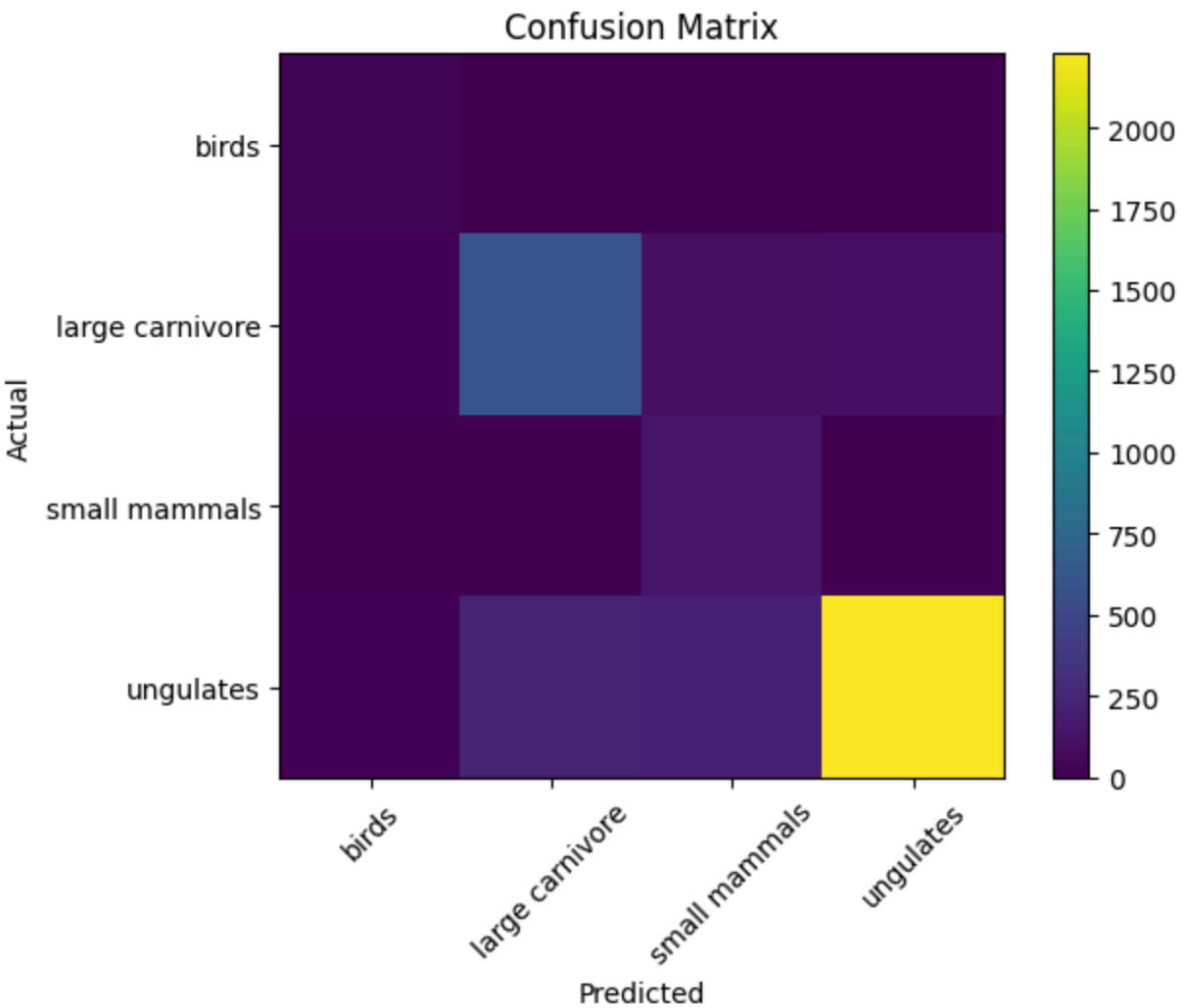
In [21]:
```
#normalized confusion matrix
cm_normalized = cm.astype(float) / cm.sum(axis=1, keepdims=True)

plt.figure(figsize=(6,5))
sns.heatmap(cm_normalized, annot=True, fmt=".2f", cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Normalized Confusion Matrix (Validation)")
plt.show()
```



In [22]:
```
cm = confusion_matrix(all_labels, all_preds)
```

```python
import matplotlib.pyplot as plt

plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()

plt.xticks(range(len(class_names)), class_names, rotation=45)
plt.yticks(range(len(class_names)), class_names)

plt.show()
```



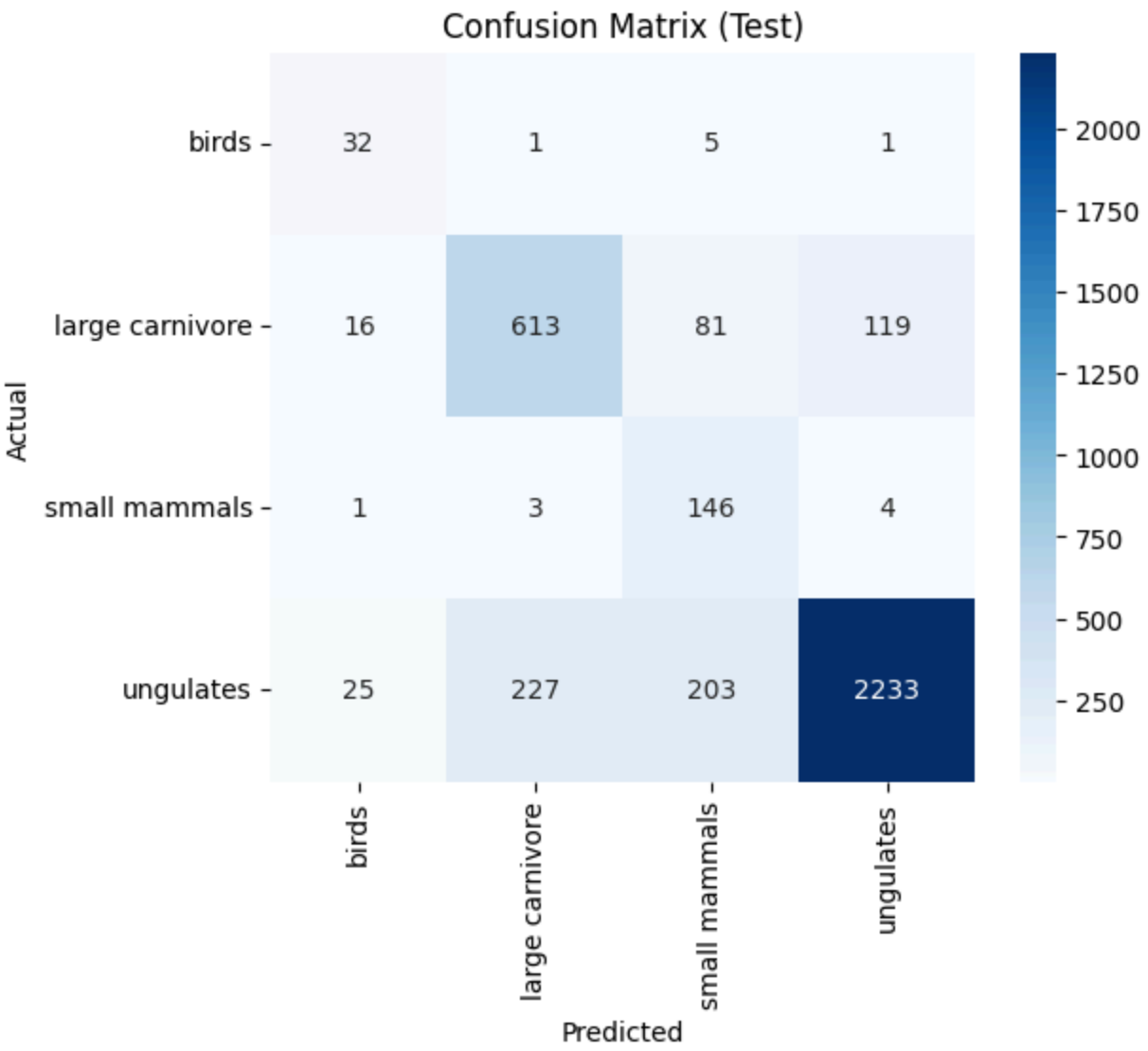
In [23]:
```
#final test and confusion matrix

model.eval()

all_test_preds = []
all_test_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
```

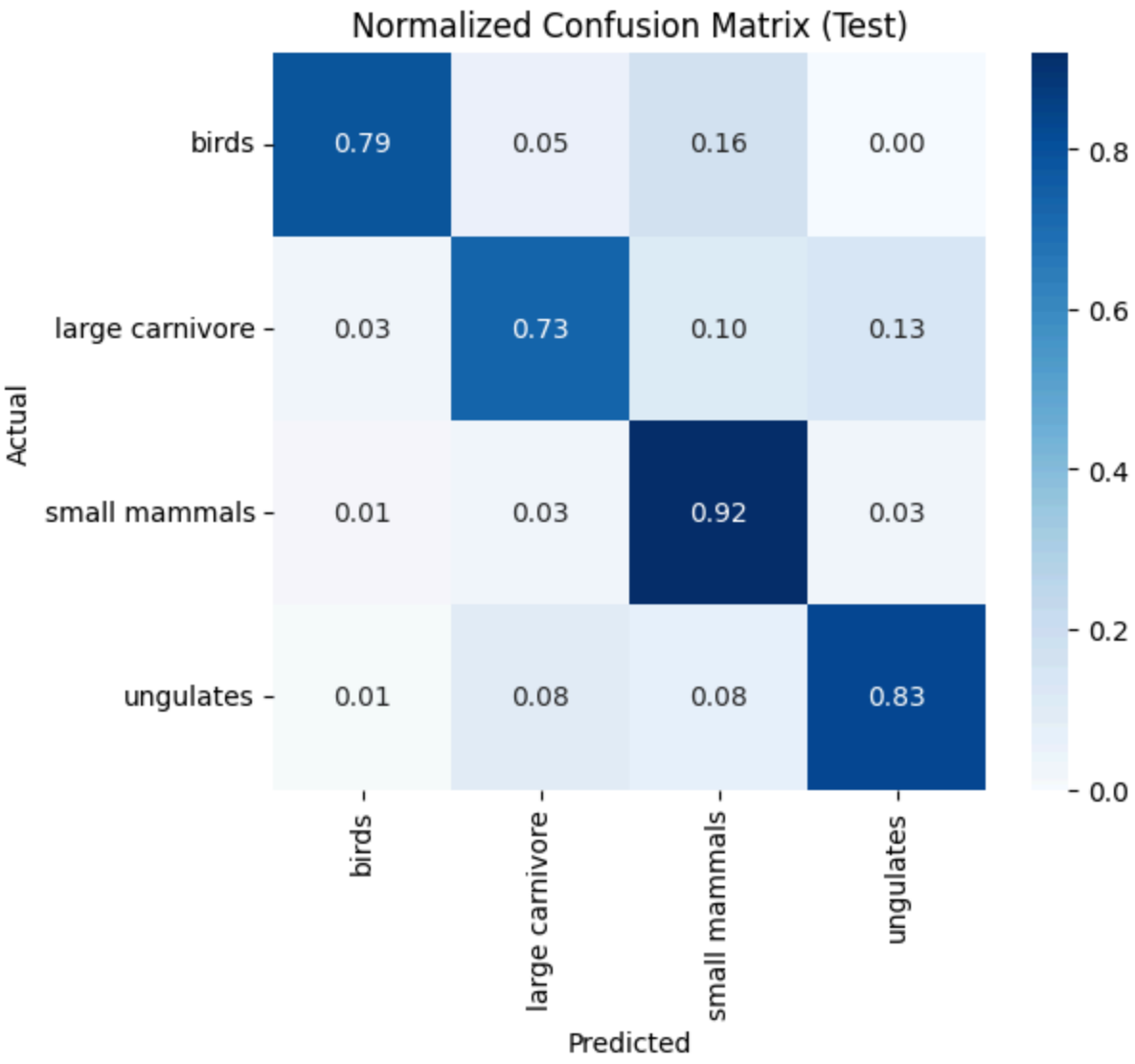
```
        labels = labels.to(device)

        outputs = model(images)
        preds = outputs.argmax(dim=1)

        all_test_preds.extend(preds.cpu().numpy())
        all_test_labels.extend(labels.cpu().numpy())

precision = precision_score(all_test_labels, all_test_preds, average='weighted')
recall = recall_score(all_test_labels, all_test_preds, average='weighted')
f1 = f1_score(all_test_labels, all_test_preds, average='weighted')
f1_macro = f1_score(all_test_labels, all_test_preds, average='macro')
accuracy = accuracy_score(all_test_labels, all_test_preds)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Macro F1 Score:", f1_macro)

cm = confusion_matrix(all_test_labels, all_test_preds)
```

```
Accuracy: 0.8150943396226416
Precision: 0.8671816738516561
Recall: 0.8150943396226416
F1 Score: 0.8316562738938943
Macro F1 Score: 0.6700438512402315
```

In [24]:

```
import seaborn as sns

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (Test)")
plt.show()
```



In [25]:
```
plt.figure(figsize=(6,5))
sns.heatmap(cm_normalized, annot=True, fmt=".2f", cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names)

plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```python
plt.title("Normalized Confusion Matrix (Test)")
plt.show()
```